THESIS OF THE END-OF-STUDY PROJECT

TO OBTAIN THE STATE ENGINEERING DIPLOMA

MAJOR: ASEDS : ADVANCED SOFTWARE ENGINEERING FOR DIGITAL SERVICES

NimbusChain Pipeline

Authored by :

Mr Mohammed KSSIM

Defended on July XX, 20XX, before the jury composed of:

	INPT	- Examiner
		- Examiner
Mr Hamid EL GHAZI	INPT	- Supervisor
Mr Hamza EL MAKRINI	Oracle	- Supervisor

AGENCE NATIONAL DE RÉGLEMENTATION DES TÉLÉCOMMUNICATIONS

INSTITUT NATIONAL DES POSTES ET TÉLÉCOMMUNICATIONS

Class of : 2023/2026

Dedicated to

*To my beloved parents, **Kebir Kssim** and **Hanane Lahboub**, words will never be enough to express the gratitude and love I carry for you in my heart. Thank you for every sacrifice you made, every prayer you whispered for me, every moment of patience, support, and unconditional love you gave throughout my journey. You taught me the values of perseverance, humility, and hard work, and everything I have achieved today is deeply rooted in your endless encouragement and belief in me. This accomplishment is as much yours as it is mine.*

*To my dear brothers and sisters, **Ranya**, **Mouaad**, **Hamza**, and **Tasnime**, thank you for being my source of strength, comfort, and joy during both the easiest and hardest moments. Your presence, your laughter, your support, and even the smallest conversations gave me the energy to keep moving forward. I am truly grateful to have grown up surrounded by such love and warmth.*

To all my high school friends, I consider myself incredibly lucky and grateful to have shared some of the most beautiful years of my life with you. The memories, laughter, challenges, and moments we lived together will always remain a special part of who I am. Thank you for the friendship, support, and unforgettable moments that stayed with me through the years.

*And finally, to all my INPT friends, everyone who was part of our countless discussions, late-night talks, laughs, and unforgettable moments in rooms **401** and **604**, thank you for making these years truly special. Those moments of friendship, support, stress, motivation, and shared dreams turned INPT into much more than just a school; they became memories and bonds that I will carry with me forever.*

Mohammed Kssim.

Acknowledgments

I would like to express my sincere gratitude to my INPT supervisor, **Mr Hamid El Ghazi**, for his guidance, availability, and valuable academic support throughout this end-of-study project. His remarks and recommendations helped me structure the work and improve both its technical and academic quality.

I would also like to warmly thank my company supervisor, **Mr Hamza El Makrini**, for his trust, his support, and his continuous guidance during my internship at Oracle. His advice, technical feedback, and knowledge of the project environment were of great help during the realization of this work.

My sincere thanks also go to the Oracle Digital Government group, and more specifically to the Data Science team, for the welcoming environment, the collaboration spirit, and the opportunity to work on the NimbusChain pipeline project in a professional and stimulating context.

I would also like to thank the entire teaching staff and administrative staff of INPT for the quality of the education and support provided throughout my engineering studies.

I would also like to express my deepest gratitude to **Fatine Mharchi** and **Driss Allaki**, for their constant help, unwavering support, and encouragement throughout my three years at INPT. Their kindness, guidance, and dedication made a meaningful difference in my academic journey, and I will always be truly grateful for their presence and support during these important years.

Finally, I would like to thank all the members of the jury for accepting to evaluate this work, as well as my family and friends for their encouragement and constant support.

Résumé

Ce projet de fin d'etudes a ete realise au sein d'Oracle Morocco R&D Center, dans le groupe Digital Government, plus precisement au sein de la Data Science team. Il porte sur le developpement et l'etude du pipeline **NimbusChain**, une plateforme destinee a l'orchestration, au traitement et a la normalisation de donnees satellitaires pour des usages d'observation de la Terre, avec un interet particulier pour le suivi agricole.

Le travail mene dans ce rapport s'inscrit dans un contexte ou les donnees issues de satellites tels que Sentinel-2 et Landsat 8/9 sont riches, mais difficiles a exploiter directement a cause de leur volume, de leur heterogeneite, de leurs formats bruts et des operations de pretraitement necessaires. L'objectif principal du projet est donc de proposer une architecture modulaire permettant de selectionner des produits satellitaires, de lancer leur traitement de maniere asynchrone, d'orchestrer les differentes etapes via une API, de convertir les scenes en jeux de donnees Zarr, et d'integrer des services de masquage afin de produire des sorties plus exploitables pour l'analyse.

Le rapport presente d'abord le contexte de l'entreprise d'accueil ainsi que le domaine de la telesurveillance spatiale appliquee a l'agriculture. Il decrit ensuite les objectifs du projet, les besoins identifies, ainsi que les principaux concepts techniques lies aux satellites utilises, a leurs bandes spectrales, a leurs niveaux de produits et a leurs systemes de tuilage. Enfin, il introduit la conception generale de la solution et son positionnement dans un environnement logiciel moderne oriente services.

Mots clés : teledetection, observation de la Terre, agriculture, Sentinel-2, Landsat, Zarr, pipeline de donnees, NimbusChain.

Summary

This end-of-study project was carried out at Oracle Morocco R&D Center within the Digital Government group, more specifically in the Data Science team. The work focuses on the **NimbusChain** pipeline, a platform designed to orchestrate, process, and normalize satellite data for Earth observation use cases, with a particular focus on agriculture-oriented analysis.

The project addresses a practical challenge: raw products delivered by satellites such as Sentinel-2 and Landsat 8/9 contain valuable information, but they are not directly convenient for operational exploitation because of their volume, heterogeneous structure, metadata complexity, and preprocessing requirements. The main objective of the project is therefore to provide a modular pipeline capable of selecting products, submitting jobs asynchronously, orchestrating the workflow through an API, converting raw scenes into Zarr datasets, and integrating masking services in order to generate outputs that are more suitable for downstream analysis.

This report first introduces Oracle and the hosting environment, then presents the remote sensing domain background necessary to understand the project. It also explains the project context, the identified needs, and the main technical concepts related to satellite missions, raw outputs, spectral bands, product levels, and tiling systems. The report finally positions the proposed solution as part of a modern service-oriented processing environment for large-scale satellite data workflows.

Key Words : remote sensing, Earth observation, agriculture, Sentinel-2, Landsat, Zarr, data pipeline, NimbusChain.

ملخص

تم إنجاز مشروع نهاية الدراسة هذا داخل مركز البحث والتطوير التابع لشركة أوراكل بالمغرب، ضمن مجموعة الحكومة الرقمية، وبالضبط داخل فريق علوم البيانات. ويتمحور هذا العمل حول مشروع مبشهين ، وهو خط معالجة مخصص لتنظيم ومعالجة وتهيئة بيانات الأقمار الصناعية من أجل استعمالها في تطبيقات رصد الأرض، مع تركيز خاص على المجال الفلاحي.

يعالج هذا المشروع إشكالية عملية تتمثل في أن المعطيات الخام القادمة من أقمار مثل صنتنل٢ و بندست ٩ غنية بالمعلومات، لكنها ليست سهلة الاستغلال بشكل مباشر بسبب حجمها الكبير، وتنوع بنيتها، وتعقيد البيانات الوصفية المرتبطة بها، والحاجة إلى مراحل تمهيدية قبل التحليل. لذلك يهدف المشروع إلى توفير بنية معيارية تسمح باختيار المنتجات الفضائية، وإطلاق المعالجة بشكل غير متزامن، وتنسيق المراحل المختلفة عبر واجهة اعي ، وتحويل المشاهد إلى صيغة ظرر ، مع دمج خدمات خاصة بالاقنعة من أجل إنتاج مخرجات أكثر ملاءمة للتحليل اللاحق.

يعرض هذا التقرير أولا بيئة الاستقبال داخل أوراكل، ثم يقدم الخلفية المفاهيمية المرتبطة بالاستشعار عن بعد اللازمة لفهم المشروع. كما يوضح سياق المشروع وأهدافه والعناصر التقنية الأساسية المرتبطة بالأقمار المستعملة، ومخرجاتها الخام، والحزم الطيفية، ومستويات المنتجات، وأنظمة التقسيم المكاني.

الكلمات المفتاحية

الاستشعار عن بعد، رصد الأرض، الفلاحة، سينتينل ٢، لاندسات، زار، خط معالجة البيانات، نيمبوس تشين

List of Figures

1.1 Oracle MADC Location 3
1.2 MADC Major Milestones 3

2.1 Sentinel-2 satellite mission. 9
2.2 Landsat 8/9 satellite mission. 10
2.3 Sentinel-2 MGRS tiling system. 14
2.4 Landsat WRS-2 path/row system. 15

List of Tables

2.1 Main components of Sentinel-2 raw outputs and their meaning. 11
2.2 Main components of Landsat 8/9 raw outputs and their meaning. 12
2.3 Main spectral band categories and their interpretation in agriculture. 13

3.1 High-level architecture of the satellite processing platform 22
3.2 Main technology stack used by the project 28

Listings

Contents

Dedication i

Acknowledgments ii

Résumé iii

Summary iv

Arabic Abstract v

List of Figures vi

List of Tables vii

List of Listings viii

Table of Contents xiii

1 Presentation of Oracle and the Project Environment 1

1.1 Introduction to Oracle 1

1.2 Oracle Corporation 2

1.3 Oracle Morocco R&D Center 2

1.4 Business Area of Activity 3

1.5 Oracle Morocco R&D Center Lines of Business 4

1.5.1 Oracle APEX 4

1.5.2 Oracle Health and Artificial Intelligence 4

1.5.3 Oracle Linux 4

1.5.4 Oracle NetSuite 5

1.5.5 Oracle Database Development Tools 5

1.5.6 Fusion ERP . . . 5
1.5.7 Oracle Analytics . . . 5
1.5.8 Oracle Cloud Infrastructure . . . 5
1.5.9 Oracle Digital Government Group . . . 6
1.5.10 Oracle Java Platform Group . . . 6
1.6 Organizational Structure . . . 6
1.7 Organizational Culture . . . 6
1.8 Conclusion . . . 7

2 Domain Background and Key Concepts - Remote Sensing 8
2.1 Earth Observation Context . . . 8
2.2 Satellites Used in the Project . . . 9
2.2.1 Sentinel-2 . . . 9
2.2.2 Landsat 8 and Landsat 9 . . . 10
2.3 Raw Satellite Outputs . . . 10
2.4 Spectral Bands and Their Meaning . . . 12
2.5 Tile System and Spatial Organization . . . 13
2.5.1 Sentinel-2 Tile System . . . 14
2.5.2 Landsat Tile System . . . 14
2.6 Satellite Product Levels . . . 15
2.7 Providers and Data Access . . . 15
2.8 Why Preprocessing Is Needed . . . 16
2.9 Conclusion . . . 16

3 General Project Context 17
3.1 Project Description . . . 18
3.2 Problem Statement . . . 18
3.3 Project Objectives . . . 19
3.4 Why Modularity Is Essential . . . 20
3.4.1 What Is Meant by Modularity . . . 20
3.4.2 Why the Services Must Be Modular . . . 20
3.5 Architectural Overview . . . 21
3.6 Description of Each Service . . . 22

3.6.1 User Interface Service . . . 22
3.6.2 API Orchestration Service . . . 23
3.6.3 Worker Service . . . 23
3.6.4 Zarr Conversion Service . . . 24
3.6.5 Mask Service . . . 24
3.6.6 Shared Contracts and Storage . . . 25
3.7 Pipeline Workflow . . . 26
3.7.1 Download and Processing Flow . . . 26
3.7.2 Multi-Tile Logic . . . 26
3.7.3 Cube Building . . . 27
3.8 Technology Stack . . . 27
3.9 Repository-Level Modularity . . . 29
3.10 Why This Architecture Is Well Adapted to the Project . . . 29
3.11 Conclusion . . . 30

4 Conception **31**
4.1 Requirements and Specifications . . . 31
4.1.1 Needs Analysis . . . 31
4.1.2 Functional Requirements . . . 31
4.1.2.1 Product Search and Preview . . . 31
4.1.2.2 Area of Interest and Tile Selection . . . 31
4.1.2.3 Asynchronous Job Submission and Monitoring . . . 31
4.1.2.4 Zarr Conversion, Masking, and Cube Generation . . . 32
4.1.3 Non-Functional Requirements . . . 32
4.1.3.1 Modularity . . . 32
4.1.3.2 Scalability . . . 32
4.1.3.3 Maintainability . . . 32
4.1.3.4 Observability and Traceability . . . 32
4.1.4 Constraints and Assumptions . . . 32
4.1.5 Conclusion . . . 32
4.2 Architecture and Technical Design . . . 32
4.2.1 Global Architecture . . . 33
4.2.2 Control Plane and Compute Plane . . . 33

4.2.3 Main Runtime Components . . . 33
4.2.3.1 User Interface Service . . . 33
4.2.3.2 API Orchestration Service . . . 33
4.2.3.3 Worker Service . . . 33
4.2.3.4 Zarr Conversion Service . . . 33
4.2.3.5 Mask Service . . . 33
4.2.3.6 Orchestration Core . . . 33
4.2.4 Persistence Layer and Shared Contracts . . . 33
4.2.5 Service-to-Service Communication . . . 34
4.2.6 Data Flow and Processing Pipeline . . . 34
4.2.6.1 Execution Control . . . 34
4.2.6.2 Pipeline States and Events . . . 34
4.2.7 Placement of Cube Generation in the Workflow . . . 34
4.2.8 Technology Stack . . . 34
4.2.9 Conclusion . . . 34

5 Implementation of the Proposed Solution 35
5.1 Implementation of the User Interface . . . 35
5.2 Implementation of the API Layer . . . 35
5.3 Implementation of the Worker and Job Execution Logic . . . 35
5.4 Implementation of the Zarr Conversion Workflow . . . 35
5.5 Implementation of the Masking Workflow . . . 35
5.6 Implementation of Multi-Tile and Cube Processing . . . 35
5.7 Artifact Management and Result Persistence . . . 35
5.8 Conclusion . . . 35

6 Validation, Results and Discussion 36
6.1 Validation Strategy . . . 36
6.2 Execution and Test Scenarios . . . 36
6.3 Observed Results . . . 36
6.4 Analysis of the Obtained Outputs . . . 36
6.5 Strengths of the Proposed Solution . . . 36
6.6 Current Limitations . . . 36

6.7 Possible Improvements 36
6.8 Conclusion 36

General Conclusion and Perspectives **37**

Chapter 1

Presentation of Oracle and the Project Environment

Introduction

This chapter presents Oracle, the professional environment in which the internship was carried out, and the organizational context surrounding the project. It introduces Oracle Corporation, highlights the Oracle Morocco Research and Development Center, summarizes the main business areas of the company, and presents the organizational structure and culture that characterize Oracle.

1.1 Introduction to Oracle

In today's world, businesses, technology companies, and public institutions rely heavily on databases to store and manage their data. They also depend on cloud services to manage their IT infrastructure, as well as on essential enterprise tools such as ERP, CRM, SCM, EPM, and SPM. These technologies now play a central role in the functioning of modern organizations, to the point that operating without them has become almost inconceivable.

Oracle, a leading American multinational technology company, not only uses these tools in its own daily operations but also designs, develops, and provides them to customers worldwide. The company has built its reputation on innovation, technological excellence, and a strong commitment to customer satisfaction.

1.2 Oracle Corporation

Oracle Corporation is a technology company headquartered in Austin, Texas. It was founded in 1977 by Larry Ellison, Bob Miner, and Ed Oates under the name *Software Development Laboratories (SDL)*.

The company initially focused on the development of Oracle Database, a relational database management system (RDBMS) that quickly became its flagship product. Over time, Oracle expanded its portfolio to include enterprise software, cloud computing solutions, hardware systems, and consulting services. One of the company's most important strategic acquisitions was Sun Microsystems in 2010, which gave Oracle access to major technologies such as Java and the Solaris operating system.

Today, Oracle is recognized as one of the leading providers of cloud-based IT infrastructure and software solutions that help organizations improve their efficiency, scale their operations, and enhance overall performance. Oracle is also known for developing the world's first autonomous database and for building Oracle Cloud as a complete cloud ecosystem.

1.3 Oracle Morocco R&D Center

The Oracle Research and Development Center Morocco is the first Oracle R&D center established in Africa and one of only a few globally. Located in Casablanca, Morocco, this software development center was inaugurated in 2022 and has become one of Oracle's key development centers in the EMEA region.

The center brings together highly skilled software developers, engineers, and project managers who contribute to a wide range of Oracle product lines. Their work makes use of recent technological trends and advanced domains such as artificial intelligence, augmented, virtual, and mixed reality, big data analytics, blockchain, cloud computing, cybersecurity, the Internet of Things, machine learning, mobile computing, and serverless computing.

The teams at Oracle MADC use these technologies to address important challenges in business, science, and the public sector. The center also supports internship opportunities, graduate recruitment initiatives, and collaborative research projects with universities in the region as part of Oracle's broader global innovation strategy.

Oracle MADC has already contributed to the delivery of several high-quality products and solutions, including Oracle Cloud Infrastructure (OCI), Autonomous Data Warehouse (ADW), and other strategic Oracle offerings.

Figure 1.1: Oracle MADC Location

Figure 1.2: MADC Major Milestones

1.4 Business Area of Activity

Oracle provides a wide range of products and services to its customers. These include software development tools, cloud services, training, consulting, and certifications. One of Oracle's best-known offerings is Oracle Database, which has maintained a leading position since 1979 and now benefits from cloud and artificial intelligence capabilities.

Oracle Cloud Infrastructure offers integrated solutions covering SaaS, PaaS, IaaS, and DaaS to meet business, IT, and development needs. Oracle Middleware also provides an integrated platform for building and running intelligent and agile applications while improving technical efficiency through modern software architectures. In addition, Oracle Services includes Oracle Advanced Customer Support Services, Oracle Premium Support, Oracle Consulting, Oracle Financing, and Oracle Managed Cloud Services.

1.5 Oracle Morocco R&D Center Lines of Business

Oracle Morocco R&D Center is organized around several Lines of Business (LOBs). In the context of a report, the detailed focus can be adapted according to the line of business to which the project belongs. The main lines of business include the following.

1.5.1 Oracle APEX

Oracle APEX is one of the most widely used enterprise low-code application platforms in the world. It enables the development of secure, scalable, and feature-rich web and mobile applications that can be deployed either on-premises or in the cloud.

With APEX, developers can quickly create and launch engaging applications that solve real business problems and provide immediate value. It is not necessary to master a large number of technologies to build sophisticated solutions, since APEX handles much of the technical complexity and allows developers to focus on solving the core problem.

1.5.2 Oracle Health and Artificial Intelligence

Oracle contributes to the transformation of healthcare technologies through data-driven and intelligent solutions. Its platforms help accelerate the development of the next generation of drugs, devices, and therapies by creating a cohesive ecosystem where data, clinical trials, strategy, and insights work together for better outcomes.

1.5.3 Oracle Linux

Oracle Linux provides the operating system of choice for Oracle customers, Oracle Cloud, and Oracle Engineered Systems. Oracle has been an active contributor to the Linux ecosystem since 1998 and delivered one of the first commercial databases on Linux along with several enhancements. Today, Oracle Linux continues to focus on open-source development while improving system stability and performance.

1.5.4 Oracle NetSuite

NetSuite is a cloud-based business management platform that helps organizations manage financial operations, business processes, and customer relationships. Its features include accounting, CRM, inventory management, e-commerce, and other integrated services within a single environment.

1.5.5 Oracle Database Development Tools

Oracle Database Development Tools provide secure data access solutions for both cloud and on-premises environments at a scale that supports modern application development. Combined with advanced data visualization and analysis capabilities, these tools support data platforms for a wide range of development needs. They are well suited to learning and applying the complete development lifecycle with current technologies and deployment models across Java and Oracle Database. Key tools include Oracle SQL Developer, Oracle Data Modeler, Oracle REST Data Services, and Oracle SQLcl.

1.5.6 Fusion ERP

Oracle Fusion Cloud ERP is a complete and modern cloud ERP suite. It offers advanced capabilities such as artificial intelligence to automate repetitive manual processes, analytics to react quickly to market changes, and automatic updates to remain current and competitive.

1.5.7 Oracle Analytics

Oracle Analytics is a comprehensive platform designed to support diverse data workloads. It transforms information into actionable insights by combining cloud, on-premises, and hybrid data sources. Through AI- and ML-powered analytical applications, it enables users to make important business decisions more quickly.

1.5.8 Oracle Cloud Infrastructure

Oracle Cloud is a public cloud platform designed to provide strong reliability, performance, and flexibility for all types of applications. By rethinking core engineering and system design for cloud computing, Oracle introduced innovations that address many limitations of traditional public cloud offerings. OCI supports enterprise workload migration and provides the services required to build modern cloud-native applications.

1.5.9 Oracle Digital Government Group

Oracle Digital Government Group develops data intelligence platforms that support government leaders in making informed decisions. Its mission is to help national governments in their digital transformation journeys and improve the quality of services delivered to citizens. The project presented in this report was carried out within this line of business, more precisely within the **Data Science team** of the Digital Government group.

1.5.10 Oracle Java Platform Group

The Oracle Java Platform Group is dedicated to the advancement and maintenance of the Java platform, one of the foundations of modern software development. This group is responsible for the development, evolution, and long-term support of the Java programming language and platform, ensuring that Java remains robust, efficient, and adaptable for a wide range of software applications.

1.6 Organizational Structure

Oracle Corporation follows a hierarchical organizational structure composed of major business divisions such as Hardware, Services, Cloud, and License. Each segment is divided into smaller business units dedicated to specific products, services, or geographic regions.

At the highest level, the Board of Directors oversees the strategic direction and performance of the company, while the Executive Leadership Team manages daily operations. Functional divisions such as sales, marketing, finance, human resources, legal, and IT operate under this executive leadership. Business units are organized by region, product, and service, with each unit typically managed by a senior vice president or a general manager.

Oracle also adopts aspects of a matrix-based organizational model in which employees may report both to a functional manager and to a business unit manager. This structure helps combine clear decision-making with cross-functional collaboration.

Overall, Oracle's organizational structure is designed to encourage teamwork, innovation, and customer focus. The hierarchical dimension supports efficient communication and governance, while the matrix dimension enables cooperation across functions and business divisions in order to achieve common goals.

1.7 Organizational Culture

Oracle has a strong corporate culture centered on innovation, customer focus, excellence, collaboration, and teamwork. The company's long history of technological progress, its emphasis on customer satisfaction, and

its commitment to maintaining a dynamic and supportive work environment all contribute to this culture.

Oracle's culture places particular emphasis on the following values:

- **Innovation**: Innovation is one of Oracle's defining characteristics. The company has a long history of creating advanced technologies and products that have shaped the technology sector. This culture is supported by strong investment in research and development and by giving employees the means and encouragement to explore new ideas and build new solutions.
- **Customer Satisfaction**: Oracle adopts a customer-centric approach to business and is committed to providing products and services that address customer needs effectively. Employees are encouraged to work closely with customers to understand their challenges and design solutions that respond to them.
- **Quality**: Oracle is committed to delivering high-quality operations, products, and services. Its reputation for reliability, performance, and technical excellence reflects this commitment. Employees are expected to strive for excellence and are supported through the tools, resources, and guidance required to succeed.
- **Collaboration and Teamwork**: Collaboration plays an essential role in achieving Oracle's objectives and delivering value to customers. The company encourages employees to work together across departments and functions, and it provides tools and practices that facilitate communication and teamwork.

Overall, Oracle's organizational culture is characterized by a clear commitment to innovation, customer focus, excellence, collaboration, and teamwork. These values are reflected in the company's products, services, and operations, and they are embraced by employees across different levels of the organization.

1.8 Conclusion

Oracle is a major global technology company whose activities cover databases, cloud computing, enterprise solutions, and development platforms. Through the Oracle Morocco R&D Center, the company has established a strong innovation presence in Morocco and in the wider EMEA region. The diversity of its business lines, its structured organization, and its culture of innovation and collaboration make Oracle a rich professional environment for carrying out complex and impactful engineering projects.

Chapter 2

Domain Background and Key Concepts - Remote Sensing

Introduction

This chapter introduces the main remote sensing concepts required to understand the project presented in this report. Since the NimbusChain pipeline operates on Earth observation data for agricultural use cases, it is important to clarify the nature of the satellite missions involved, the structure of their raw products, the meaning of their spectral bands, the logic of their tile systems, and the reasons why preprocessing is necessary before analysis-ready outputs can be produced.

2.1 Earth Observation Context

Earth observation refers to the acquisition of information about the Earth's surface through remote sensing technologies, especially satellites. In the context of agriculture, Earth observation plays an important role because it makes it possible to monitor large cultivated areas repeatedly and objectively without relying only on field visits.

Agricultural monitoring benefits from satellite imagery in several ways. It can support crop growth monitoring, vegetation condition assessment, seasonal evolution tracking, irrigation analysis, stress detection, land-use mapping, and the observation of phenomena such as drought impacts or abnormal field behavior. Because agricultural areas evolve over time and often cover large regions, satellite data provides a practical way to capture both spatial and temporal variability.

The interest of Earth observation for this project is therefore directly linked to the need for automated access,

processing, and preparation of large quantities of satellite imagery. The NimbusChain pipeline is designed to make these data easier to retrieve, organize, transform, and prepare for downstream exploitation.

2.2 Satellites Used in the Project

The project mainly relies on two Earth observation families: **Sentinel-2** and **Landsat 8/9**. These missions are widely used in optical remote sensing and are particularly relevant for agricultural and environmental applications.

2.2.1 Sentinel-2

Sentinel-2 is part of the European Copernicus program and is operated by the European Space Agency. The mission is dedicated to high-resolution optical imaging for land monitoring. It is especially useful for agriculture, vegetation analysis, land-cover mapping, and environmental observation.

Sentinel-2 is known for its multispectral capability and its relatively high revisit frequency. The mission provides imagery in 13 spectral bands with spatial resolutions of 10 m, 20 m, and 60 m depending on the band. This diversity makes Sentinel-2 particularly suitable for vegetation analysis because it includes visible bands, near-infrared bands, shortwave infrared bands, and red-edge bands that are highly useful for crop monitoring.

Figure 2.1: Sentinel-2 satellite mission.

2.2.2 Landsat 8 and Landsat 9

Landsat 8 and Landsat 9 belong to the Landsat program led by NASA and the USGS. These missions are among the most important long-term Earth observation programs because they provide continuous medium-resolution satellite imagery of the Earth's surface over several decades.

Landsat 8 and 9 carry optical and thermal sensors. Their products are widely used for land monitoring, agricultural mapping, water resource analysis, vegetation studies, and environmental change detection. The multispectral bands are generally delivered at 30 m spatial resolution, the panchromatic band at 15 m, and the thermal information at coarser native resolution. Compared with Sentinel-2, Landsat offers slightly lower spatial resolution for optical bands, but it remains a major reference source because of its historical depth, reliability, and wide scientific use.

Figure 2.2: Landsat 8/9 satellite mission.

2.3 Raw Satellite Outputs

The raw outputs handled by remote sensing workflows are not simple image files. A satellite product is generally a structured set of raster files, metadata, quality information, georeferencing elements, and product descriptors. These outputs are designed for scientific and operational exploitation, but they are not always directly convenient for application pipelines or end users.

For Sentinel-2, products are typically distributed as structured archives containing metadata files, band images, quality layers, and granule-level organization. The product may include several spatial resolutions and a hierarchy that separates scene metadata from tile-specific content. For Landsat 8/9, products are usually delivered as collections of band files accompanied by metadata descriptors and quality assessment information.

From a processing perspective, the raw output may therefore contain spectral bands, acquisition metadata, product identifiers, cloud-related information, projection parameters, and multiple auxiliary files. This complexity explains why an intermediate transformation stage is often necessary before the data can be used efficiently in an automated platform.

Sentinel-2 raw output element	**Meaning and role**
Product metadata file	Describes the product globally, including acquisition information, sensing time, processing level, and general scene descriptors.
Granule or tile identifier	Identifies the MGRS spatial unit covered by the product and links the scene to its tile-level organization.
Spectral band images	Store the measured values for the Sentinel-2 multispectral bands at 10 m, 20 m, or 60 m resolution.
Quality layers	Provide scene-quality information related to clouds, defective pixels, and auxiliary masks used during interpretation or preprocessing.
Georeferencing information	Specifies the projection and positioning elements needed to locate the imagery correctly on the Earth's surface.
Auxiliary files	Contain complementary technical information used to organize, validate, or further process the product.

Table 2.1: Main components of Sentinel-2 raw outputs and their meaning.

Landsat 8/9 raw output element	**Meaning and role**
Metadata text file	Contains acquisition date, product identifiers, processing details, projection metadata, and scene-level descriptive parameters.
Multispectral band files	Store the optical observations for the visible, near-infrared, and shortwave infrared bands, generally at 30 m spatial resolution.
Panchromatic band	Provides a higher spatial resolution band, typically used for sharpening or enhanced visual interpretation.
Thermal band data	Contains thermal information that can support temperature-related and environmental analyses.
Quality assessment band	Encodes information about clouds, shadows, water, snow, and other quality indicators affecting the scene.
Path/row reference	Identifies the WRS-2 acquisition unit and links the product to Landsat orbital scene organization.

Table 2.2: Main components of Landsat 8/9 raw outputs and their meaning.

2.4 Spectral Bands and Their Meaning

Satellite sensors do not only capture images in the visible domain. They measure reflectance or radiance in several spectral bands, each one carrying different information about the observed surface. This is one of the main reasons why remote sensing is so useful in agriculture.

The visible bands, namely blue, green, and red, provide information that is close to human visual perception. They are useful for general interpretation, land-cover distinction, and the visual assessment of scenes. The near-infrared band is particularly important for vegetation studies because healthy vegetation strongly reflects in this part of the spectrum. Shortwave infrared bands are useful for moisture-related analysis, soil and vegetation condition assessment, and the detection of surface properties that are less visible in the standard RGB view.

Sentinel-2 also provides red-edge bands, which are especially valuable for agricultural applications because they can capture subtle variations in vegetation condition and canopy properties. Landsat 8/9 complements optical observation with thermal information, which can be useful for temperature-related analysis and environmental interpretation.

In practice, multispectral data allows the computation of indicators such as vegetation indices, water-related indices, and other analytical products that are not accessible from a standard natural-color image alone.

Band category	**Spectral region**	**Interpretation in agriculture**
Blue band	Visible blue	Useful for general scene interpretation and can contribute to water and atmospheric analysis.
Green band	Visible green	Supports vegetation observation and natural-color rendering of agricultural fields.
Red band	Visible red	Important for vegetation contrast and widely used in crop condition analysis.
Near-infrared band	NIR	Strongly reflected by healthy vegetation and therefore very useful for vigor and biomass assessment.
Red-edge bands	Red-edge	Particularly relevant for detecting subtle variations in crop condition, canopy structure, and chlorophyll-related behavior.
Shortwave infrared bands	SWIR	Useful for moisture-related interpretation, soil and vegetation condition analysis, and stress detection.
Thermal bands	Thermal infrared	Used mainly with Landsat for temperature-related interpretation and environmental or water-stress analysis.

Table 2.3: Main spectral band categories and their interpretation in agriculture.

2.5 Tile System and Spatial Organization

Satellite products are generally not distributed as one continuous global image. Instead, they are organized according to spatial partitioning systems that make storage, distribution, and processing more manageable. Understanding these systems is essential because the area of interest selected by a user may intersect multiple

spatial units, which directly affects search, download, and downstream processing.

2.5.1 Sentinel-2 Tile System

Sentinel-2 uses a tile system based on the Military Grid Reference System (MGRS). In practical terms, products are distributed as geographic tiles that represent fixed areas on the Earth's surface. A given agricultural region may therefore overlap one or several Sentinel-2 tiles. This is particularly important for a platform such as NimbusChain because a single user request may need to retrieve and process multiple tiles before complete coverage is achieved.

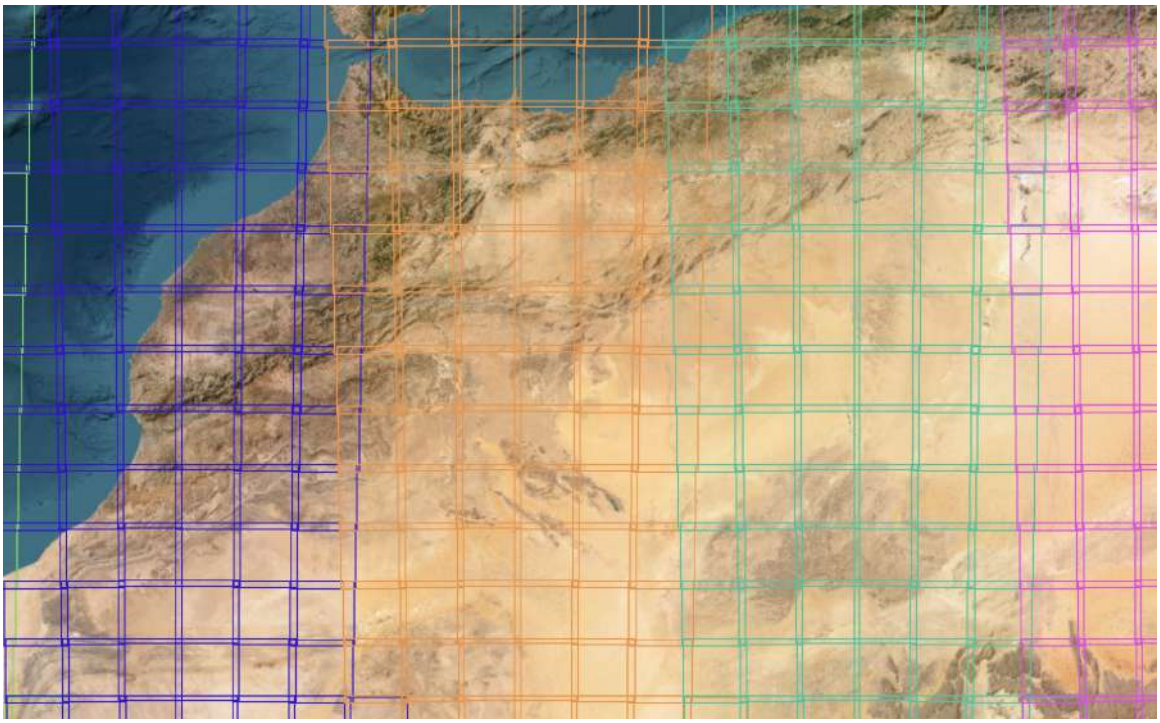

Figure 2.3: Sentinel-2 MGRS tiling system.

2.5.2 Landsat Tile System

Landsat products are organized according to the Worldwide Reference System, commonly referred to as WRS-2. This system is based on orbital acquisition geometry and identifies scenes through a *path/row* logic. Unlike a simple regular map grid, the Landsat organization is tied to the satellite's acquisition pattern. As a result, users and processing services must reason in terms of path and row combinations rather than only cartographic tiles.

These two systems differ in structure, naming, and acquisition logic. This difference is one of the reasons why provider-specific handling is necessary in the pipeline, especially for product search, metadata interpretation, and multi-scene processing.

Figure 2.4: Landsat WRS-2 path/row system.

2.6 Satellite Product Levels

Satellite missions generally provide several product levels corresponding to different stages of correction and preparation. These levels indicate how far the data has progressed from sensor acquisition toward directly usable geophysical information.

For Sentinel-2, one of the most common distinctions is between **Level-1C** and **Level-2A**. Level-1C products represent top-of-atmosphere reflectance with geometric correction, while Level-2A products provide bottom-of-atmosphere reflectance after atmospheric correction. For agricultural analysis, Level-2A is often preferred because it is more suitable for surface-oriented interpretation.

For Landsat 8/9, a similar distinction exists between products that are closer to radiometrically and geometrically corrected acquisition data and products that already include higher-level surface information such as surface reflectance and related derived outputs. Understanding these product levels is important because the nature of the downstream preprocessing depends partly on the level at which the data enters the pipeline.

2.7 Providers and Data Access

The project interacts with external providers rather than storing all source imagery internally from the beginning. In practice, data access depends on the mission and on the service that distributes its products.

Sentinel-2 data is accessed through the Copernicus ecosystem, while Landsat 8/9 data is distributed through USGS-related services. Although both sources provide Earth observation imagery, they differ in authentication mechanisms, metadata structure, product naming, query semantics, and download behavior. These differences create an integration challenge for any unified platform.

From the point of view of NimbusChain, this means that product search and retrieval cannot rely on a single universal logic. The platform must instead handle provider-specific APIs, credentials, metadata formats, and result interpretation rules while still exposing a coherent experience to the final user.

2.8 Why Preprocessing Is Needed

Preprocessing is a necessary step because raw satellite products are rarely ready for direct analytical use in an operational application. Several issues must be addressed before the data can be exploited efficiently.

First, the raw products come from different providers and therefore differ in structure, naming, metadata, and scene organization. Second, agricultural analysis often requires harmonized and directly accessible numerical arrays rather than heterogeneous raw files. Third, optical imagery is frequently affected by clouds, cloud shadows, water-related artifacts, or scene quality variability, which can reduce the relevance of direct interpretation. Fourth, the same area of interest may involve multiple scenes or tiles that must later be organized consistently in space and time.

For these reasons, preprocessing in the NimbusChain pipeline includes operations such as normalization, structured storage conversion, masking, and preparation for cube generation. In this project, the use of Zarr is particularly important because it transforms raw and heterogeneous scene outputs into a format that is better suited for scalable access, array-based computation, and multi-scene exploitation.

2.9 Conclusion

Remote sensing data is rich and highly informative, but it is also structurally complex. Understanding the missions used in the project, the nature of their raw products, the meaning of their spectral information, the logic of their tiling systems, and the role of preprocessing is essential for understanding the design of the NimbusChain pipeline. This background explains why the platform requires modular acquisition, normalization, masking, and structured storage components before higher-level analysis can be performed.

Chapter 3

General Project Context

Introduction

The project presented in this report is called the **NimbusChain pipeline**. It is an automated and modular satellite imagery processing platform. Its role is not limited to downloading remote sensing products. Instead, it implements a complete processing pipeline that starts with data discovery and acquisition, continues with asynchronous job orchestration, and extends to downstream preparation stages such as Zarr conversion, cloud and water masking, and multi-scene cube generation.

This type of system answers a real need in Earth observation projects. In practice, users do not simply request one file and stop there. They define an area of interest, choose a provider and a product family, filter scenes by date, inspect candidate products, execute long-running downloads, and then prepare the resulting data for later exploitation. When the spatial extent intersects several tiles, the workflow becomes even more demanding because the platform must manage multiple related jobs while keeping execution traceable, reliable, and scalable.

For these reasons, the project adopts a service-oriented and modular architecture. Rather than concentrating all responsibilities inside one monolithic application, the platform separates the user interface, orchestration API, worker execution logic, Zarr conversion runtime, and masking runtime into explicit components. This design improves maintainability, observability, scalability, and future extensibility while supporting an automated end-to-end workflow from raw data acquisition to analysis-ready outputs.

This work was carried out within Oracle's **Digital Government (DGov)** line of business, and more specifically within the **Data Science team** of that group. The development of the NimbusChain pipeline is therefore part of a broader effort to build data-driven solutions that support digital government use cases.

3.1 Project Description

The NimbusChain pipeline can be described as an end-to-end satellite data acquisition and preparation system. It is designed to transform raw Earth observation products into structured and directly usable outputs through a unified processing chain. In operational terms, it allows a user to:

- Define an Area of Interest (AOI),
- Preview products from external providers,
- Submit one or several jobs for asynchronous execution,
- Track the progress of running jobs,
- Convert raw downloaded scenes into normalized Zarr datasets,
- Apply cloud and water masks on generated Zarr outputs,
- Build grouped or single spatio-temporal cubes from converted scenes.

The studied implementation goes beyond a simple downloader. It introduces a real processing pipeline in which each stage produces structured artifacts that can be reused by the next one. Raw scenes are first acquired from providers such as Copernicus or USGS, then persisted locally, converted into analysis-friendly Zarr stores, optionally enriched with cloud and water masks, and finally assembled into cubes when a multi-scene time series is required.

This means the project should be understood as a **satellite data pipeline platform** rather than a basic file transfer application. Its value lies in automation, modularity, and the ability to produce reliable and reusable geospatial outputs from heterogeneous remote sensing inputs.

3.2 Problem Statement

Satellite product handling is more complex than a traditional download task. Several technical difficulties must be addressed at the same time:

- Different providers expose different APIs, credentials, and metadata conventions,
- One area of interest may intersect multiple acquisition tiles,
- Download operations may be long, unstable, or partially successful,

- Users need visibility into job states, errors, artifacts, and final outputs,
- Raw products are not always directly usable for analysis,
- Downstream processing often requires normalized formats and quality masks,
- The architecture must remain extensible for future providers and services.

If all these concerns are mixed into a single script, the result quickly becomes difficult to maintain and difficult to scale. The project therefore solves the problem by organizing the platform as a set of cooperating services, each one responsible for a specific part of the workflow, from acquisition and orchestration to conversion and quality-improvement stages.

3.3 Project Objectives

The main objectives of the project can be summarized as follows:

- Provide a practical interface for selecting and previewing satellite products,
- Automate the progression from product selection to processed outputs,
- Orchestrate jobs through a clean API instead of direct manual execution,
- Process requests asynchronously through background workers,
- Support multi-tile and multi-scene workflows,
- Normalize raw remote sensing products into Zarr datasets,
- Improve data usability by integrating masking services,
- Prepare outputs that are better suited for downstream analysis,
- Make the platform easy to deploy locally and in containerized environments,
- Keep the codebase extensible through modular design.

3.4 Why Modularity Is Essential

3.4.1 What Is Meant by Modularity

In software engineering, **modularity** means decomposing a system into independent and coherent units, where each unit has a clear responsibility and communicates with the others through well-defined interfaces.

In this project, modularity does not simply mean splitting files into folders. It means designing the platform so that:

- The user interface is responsible for interaction and visualization,
- The API is responsible for validation, orchestration, and public endpoints,
- The worker is responsible for execution of long-running jobs,
- The Zarr service is responsible for data normalization and conversion,
- The mask service is responsible for cloud and water masking,
- Shared contracts define how services exchange structured information.

In other words, each module or service should do one family of tasks well, without taking over the responsibilities of the others.

3.4.2 Why the Services Must Be Modular

This project is a strong example of why modularity matters in real projects. The platform handles user interaction, provider communication, background execution, geospatial data conversion, inference-based masking, artifact management, and cube building. These concerns are too different to be efficiently maintained in one tightly coupled component.

Modularity is necessary here for several reasons:

- **Maintainability**: Each service can evolve without rewriting the whole platform.
- **Scalability**: Worker capacity can be increased independently from the UI.
- **Fault isolation**: A conversion or masking issue does not mean the UI must fail.
- **Extensibility**: New providers, new processing stages, or new deployment modes can be added with limited impact.

- **Testability**: API logic, worker logic, conversion logic, and UI logic can be tested separately.
- **Operational clarity**: Each runtime component exposes a clearly identifiable role.
- **Reuse**: Shared contracts and service clients can be reused by several components.

For a satellite workflow platform, modularity is therefore not only a design preference. It is a practical condition for building a robust and extensible system.

3.5 Architectural Overview

The studied implementation is organized around five main runtime components:

1. A user-facing Streamlit application,
2. A public FastAPI orchestration API,
3. A background worker responsible for executing jobs,
4. A standalone Zarr conversion service,
5. A standalone masking service.

These components exchange structured information through HTTP contracts and through shared persistent storage. At a high level, the workflow can be summarized as follows:

1. The user prepares a request in the UI,
2. The UI sends the request to the API,
3. The API validates and stores the job,
4. The worker claims and executes the job,
5. The worker downloads raw products from external providers,
6. The worker triggers conversion and optional masking,
7. Artifacts and results are registered and exposed back to the user.

Component	Technology	Main responsibility
User interface	Streamlit	Collect user inputs, preview products, submit jobs, and monitor execution.
Public API	FastAPI	Expose the control plane for jobs, artifacts, health, events, conversion, and masking actions.
Worker runtime	Python async worker	Poll the job store, execute downloads, and drive the pipeline lifecycle.
Zarr service	FastAPI service	Convert raw satellite scenes into normalized Zarr outputs and build cubes.
Mask service	FastAPI service	Apply cloud and water masks to existing Zarr outputs.
Persistence layer	MongoDB or SQLite	Store job records, events, states, artifacts, and worker runtime information.

Table 3.1: High-level architecture of the satellite processing platform

3.6 Description of Each Service

3.6.1 User Interface Service

The user interface is implemented with **Streamlit**. It is the entry point of the platform from the user perspective. Its role is to simplify complex geospatial and operational tasks through an interactive interface.

The UI supports several important operations:

- AOI definition and editing,
- Tile discovery and tile selection,
- Preview of candidate products,
- Job submission,
- Monitoring of active and completed jobs,
- Access to Zarr and artifact utilities,

- Display of runtime health and processing status.

An important design choice is that the UI does not execute downloads directly. Instead, it acts as a client of the API. This is a modularity principle in action: the interface handles interaction, while execution is delegated to backend components.

3.6.2 API Orchestration Service

The API service is implemented with **FastAPI**. It is the main public backend surface of the platform and acts as the control plane.

Its responsibilities include:

- Validating incoming job requests,
- Creating, listing, filtering, and inspecting jobs,
- Exposing job events and health information,
- Registering artifacts and output metadata,
- Exposing worker status,
- Exposing converter and mask endpoints under the `/v1` namespace,
- Providing a stable contract to the UI and external callers.

The API does not perform heavy remote sensing processing itself. This separation keeps the orchestration layer responsive even when long-running jobs are in progress.

3.6.3 Worker Service

The worker is the execution engine of the platform. It continuously polls the configured job store, claims queued jobs, and runs the processing pipeline in the background.

Its responsibilities include:

- Reading queued jobs from the store,
- Claiming jobs atomically,
- Launching provider-specific search and download logic,

- Persisting raw outputs,
- Triggering Zarr conversion,
- Coordinating mask application when requested,
- Updating events, state transitions, and result payloads,
- Reporting runtime heartbeats and worker capacity.

Because the worker is separated from the API and UI, it can be scaled independently. This is especially valuable when multiple jobs or multiple tiles must be processed concurrently.

3.6.4 Zarr Conversion Service

The Zarr service is a dedicated processing component responsible for transforming downloaded raw products into structured Zarr datasets. This is a major added value of the branch under study because it moves the platform from data acquisition to data preparation.

Its responsibilities include:

- Reading raw scenes or archives from storage,
- Preserving native imagery layers,
- Organizing outputs into normalized Zarr layouts,
- Generating ancillary arrays when needed,
- Exposing health, readiness, schema, and conversion endpoints,
- Supporting grouped and single cube building from existing Zarr scenes.

The use of a standalone service for conversion is important. It allows the conversion logic to evolve independently from the downloader and makes the architecture more suitable for future growth.

3.6.5 Mask Service

The mask service is another dedicated backend component. Its role is to apply cloud and water masking on already generated Zarr outputs.

Its responsibilities include:

- Receiving structured masking requests,
- Loading the source Zarr dataset,
- Applying cloud and/or water masking inference,
- Writing mask layers or masked outputs,
- Exposing health and schema endpoints,
- Returning masking metadata to the pipeline.

This service is particularly useful because raw satellite data often contains unusable or low-quality pixels. By integrating masking as an explicit service, the platform improves the analytical quality of downstream outputs.

3.6.6 Shared Contracts and Storage

The architecture also relies on a shared package dedicated to contracts, clients, and common conventions. This package allows services to exchange structured payloads rather than loosely defined messages. The project also uses persistent storage, either **MongoDB** or **SQLite**, to store jobs, events, artifacts, results, and worker states.

This shared layer is essential because the services do not exchange large raster data directly inside HTTP payloads. Instead, they exchange references such as:

- `job_id`,
- `raw_uri`,
- `output_uri`,
- `zarr_uri`,
- `source_zarr_uri`,
- `scene_id`.

This design reduces coupling and avoids transferring heavy artifacts through inappropriate channels.

3.7 Pipeline Workflow

3.7.1 Download and Processing Flow

The complete pipeline can be described in successive stages:

1. The user defines an AOI, date range, provider, and product configuration,
2. The UI sends a request to the API,
3. The API stores the job in the configured backend,
4. The worker claims the job and starts execution,
5. External providers return metadata and downloadable products,
6. Raw products are written to shared storage,
7. The worker triggers Zarr conversion,
8. Converted outputs are registered as artifacts,
9. Optional masking is applied on the produced Zarr store,
10. Final results, paths, events, and summaries are exposed to the UI.

This flow illustrates a key property of the platform: it is built as a **pipeline**, not as a one-step downloader.

3.7.2 Multi-Tile Logic

One of the most important capabilities of the selected branch is its support for **multi-tile** workflows. In Earth observation, a single AOI may intersect several acquisition tiles. Handling this case correctly is crucial.

Instead of treating the whole request as one opaque download, the platform can decompose the request into a set of jobs associated with selected tiles. This has several advantages:

- Each tile becomes an explicit processing unit,
- Failures can be isolated more easily,
- Progress tracking becomes more precise,
- Worker resources can process jobs concurrently,

- Downstream conversion and cube generation can be organized more cleanly.

This branch extends the architectural logic from job orchestration toward **multi-scene and multi-tile data preparation**.

3.7.3 Cube Building

Another major feature of the branch is the integration of **cube generation**. After scenes are converted into Zarr format, they can be assembled into temporal cubes. The repository supports both single-cube and grouped cube construction.

From an engineering point of view, this means the platform is no longer limited to collecting raw data. It also starts preparing datasets in a form that is closer to analytical and machine learning workloads. This evolution significantly increases the value of the system.

3.8 Technology Stack

The project relies on a modern Python-based stack chosen to support service modularity, asynchronous execution, geospatial processing, and deployability.

Category	Technology	Role in the project
Programming language	Python 3.11	Main implementation language for the complete platform.
Public backend framework	FastAPI	API development, validation, health endpoints, and internal processing services.
Frontend layer	Streamlit	Interactive operational interface for request preparation and monitoring.
Data validation	Pydantic and Pydantic Settings	Validation of DTOs, request payloads, and runtime configuration.
Async and HTTP communication	AnyIO, AioHTTP, HTTPX	Asynchronous execution and service-to-service communication.
Geospatial tooling	Shapely, GeoPandas, PyProj	AOI handling, geometry operations, and spatial utilities.
Raster and array processing	Rasterio, Xarray, Zarr, NumCodecs, H5Py, H5netCDF	Reading products, array transformation, and normalized storage creation.
Persistence	MongoDB and SQLite	Job store, event persistence, artifact metadata, and runtime coordination.
Observability	Prometheus Client and logging utilities	Metrics, health monitoring, and runtime diagnostics.
Inference-related masking stack	FastAI, OmniCloudMask, OmniWaterMask	Cloud and water masking on generated Zarr outputs.
Application server	Uvicorn	Service runtime for FastAPI applications.
Code quality and testing	Pytest and Ruff	Verification and quality control.
Containerization and deployment	Docker, Podman, Compose, Kubernetes	Local multi-service execution and cluster-oriented deployment.

Table 3.2: Main technology stack used by the project

3.9 Repository-Level Modularity

The modularity of the project is visible not only at runtime, but also in the repository structure. The source code is divided into specialized packages:

- `nimbuschain_fetch`: Core orchestration logic, providers, worker, stores, and workflows,
- `nimbuschain_fetch_service`: The public API layer,
- `nimbuschain_fetch_ui`: The Streamlit interface,
- `nimbuschain_zarr_service`: Conversion and cube-building logic,
- `nimbuschain_mask_service`: Cloud and water masking logic,
- `nimbuschain_shared`: Shared clients, contracts, and runtime helpers.

This structure shows that modularity is applied consistently at two levels:

- At the **runtime level**, where services run as independent components,
- At the **codebase level**, where source packages are organized by responsibility.

3.10 Why This Architecture Is Well Adapted to the Project

The architecture selected in this project is well adapted to satellite workflows because it reflects the actual lifecycle of the data:

- Product discovery is separated from heavy execution,
- Execution is separated from post-processing,
- Post-processing is separated into conversion and masking concerns,
- Cube generation builds on already normalized outputs,
- Persistent storage guarantees traceability across the whole pipeline.

This separation creates a system that is easier to understand, easier to operate, and easier to extend. A future developer can add a new provider, a new artifact type, or a new processing stage without redesigning the entire application.

3.11 Conclusion

The project studied in this report is a modular satellite data pipeline platform that combines acquisition, orchestration, monitoring, conversion, masking, and cube generation. The selected implementation demonstrates a clear evolution toward a more complete and production-oriented architecture. Its main strength lies in the explicit separation of responsibilities between services and in the use of structured contracts to connect them.

For the rest of this report, this general context is important because it explains the technical choices made throughout the implementation. The platform was designed not only to fetch satellite data, but to prepare it in a reliable and extensible way for downstream scientific and engineering use.

Chapter 4

Conception

Introduction

4.1 Requirements and Specifications

4.1.1 Needs Analysis

4.1.2 Functional Requirements

4.1.2.1 Product Search and Preview

4.1.2.2 Area of Interest and Tile Selection

4.1.2.3 Asynchronous Job Submission and Monitoring

4.1.2.4 Zarr Conversion, Masking, and Cube Generation

4.1.3 Non-Functional Requirements

4.1.3.1 Modularity

4.1.3.2 Scalability

4.1.3.3 Maintainability

4.1.3.4 Observability and Traceability

4.1.4 Constraints and Assumptions

4.1.5 Conclusion

4.2 Architecture and Technical Design

Introduction

4.2.1 Global Architecture

4.2.2 Control Plane and Compute Plane

4.2.3 Main Runtime Components

4.2.3.1 User Interface Service

4.2.3.2 API Orchestration Service

4.2.3.3 Worker Service

4.2.3.4 Zarr Conversion Service

4.2.3.5 Mask Service

4.2.3.6 Orchestration Core

4.2.4 Persistence Layer and Shared Contracts

4.2.5 Service-to-Service Communication

4.2.6 Data Flow and Processing Pipeline

4.2.6.1 Execution Control

4.2.6.2 Pipeline States and Events

4.2.7 Placement of Cube Generation in the Workflow

4.2.8 Technology Stack

4.2.9 Conclusion

Chapter 5

Implementation of the Proposed Solution

Introduction

This chapter focuses on how the designed architecture was implemented in practice, with attention to the main modules, services, and execution workflows.

5.1 Implementation of the User Interface

5.2 Implementation of the API Layer

5.3 Implementation of the Worker and Job Execution Logic

5.4 Implementation of the Zarr Conversion Workflow

5.5 Implementation of the Masking Workflow

5.6 Implementation of Multi-Tile and Cube Processing

5.7 Artifact Management and Result Persistence

5.8 Conclusion

Chapter 6

Validation, Results and Discussion

Introduction

This chapter presents how the platform was validated, the observed results, and the main strengths and limitations identified during the project.

6.1 Validation Strategy

6.2 Execution and Test Scenarios

6.3 Observed Results

6.4 Analysis of the Obtained Outputs

6.5 Strengths of the Proposed Solution

6.6 Current Limitations

6.7 Possible Improvements

6.8 Conclusion

General Conclusion and Perspectives

This final chapter summarizes the overall contribution of the project, highlights the main technical achievements, and presents future perspectives for improving and extending the platform.